Python Activity Based Learning

Problem-1: Examination Result and Grace-Mark Validation.

Problem Statement: Develop a Python program to accept marks in three subjects, check the subject-wise pass criteria, apply grace marks when eligible, and display the final examination result.

Objective:

- To evaluate students' examination results using conditional statements and a specified grace-mark policy.

Assumptions:

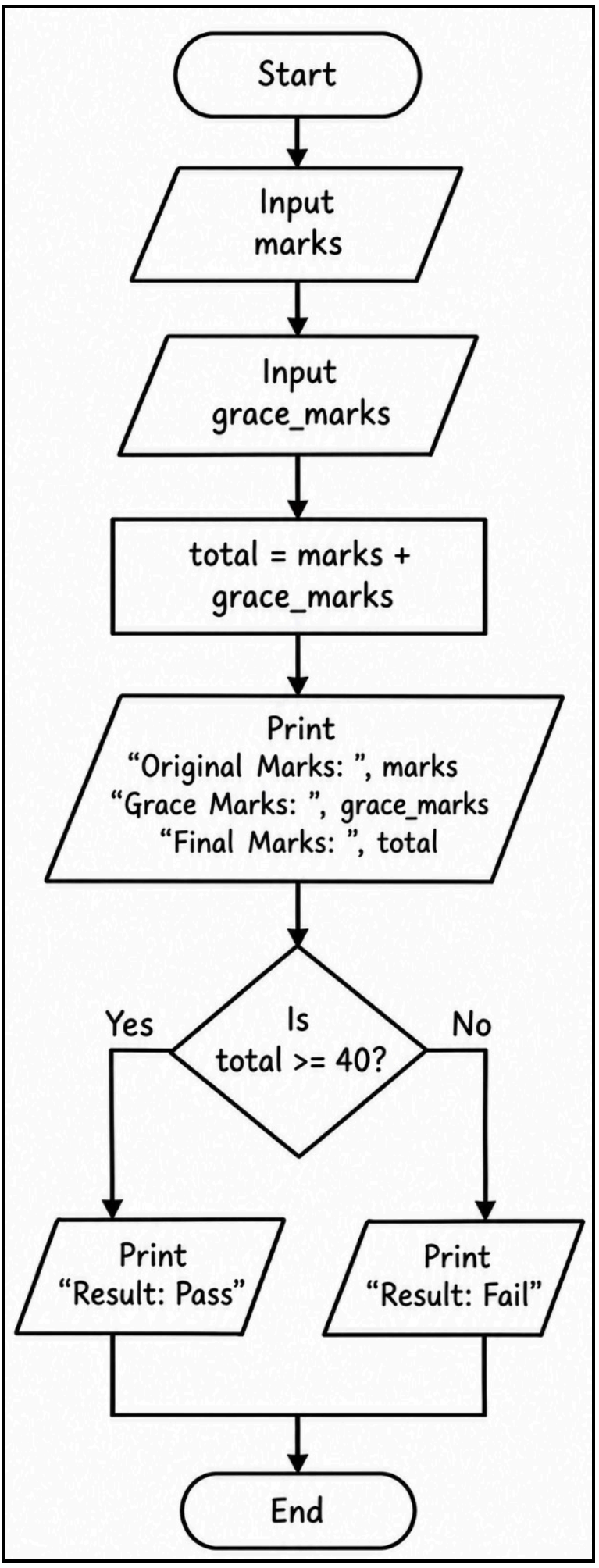
- There are three subjects.
- Maximum marks in each subject are 100.
- Minimum pass marks are 35.
- A maximum of 5 grace marks can be given to only one failed subject.
- Grace marks are applicable only when the student fails exactly one subject and needs no more than 5 additional marks.
- Marks must be between 0 and 100.

Algorithm:

1. Start.
2. Input marks for three subjects.
3. Check whether all marks are between 0 and 100.
4. If any mark is invalid, display an error.
5. Otherwise, identify subjects with marks below 35.
6. If no subject is failed, display PASS.
7. If exactly one subject is failed by 5 marks or less, apply grace marks.

8. Check the result again.
9. Display the final marks and result.
10. Stop.

Flowchart:


Start
Input marks
Input grace_marks
total = marks + grace_marks
Print "Original Marks: ", marks "Grace Marks: ", grace_marks "Final Marks: ", total
Is total >= 40?
Yes
No
Print "Result: Pass"
Print "Result: Fail"
End

Psuedocode:

```
text
START
INPUT marks of three subjects

IF any mark is below 0 or above 100
   DISPLAY "Invalid marks"
ELSE
   FIND subjects with marks below 35

   IF no subject failed
      DISPLAY "PASS"
   ELSE IF exactly one subject failed
      IF the difference from 35 is at most 5
         Increase that subject's marks to 35
         DISPLAY "PASS with grace marks"
      ELSE
         DISPLAY "FAIL"
   ELSE
      DISPLAY "FAIL"
   END IF

   DISPLAY final marks
END IF
STOP
```

Program:

```
marks = int(input("Enter your marks: "))
grace_marks = int(input("Enter grace marks: "))

total = marks + grace_marks

print("Original Marks:", marks)
print("Grace Marks:", grace_marks)
```

```
print("Final Marks:", total)

if total >= 40:
    print("Result: Pass")
else:
    print("Result: Fail")
```

Test Cases:

Case-1:

```
Enter grace marks: 75
Original Marks: 90
Grace Marks: 75
Final Marks: 165
Result: Pass
```

Case-2:

```
Enter your marks: 85
Enter grace marks: 75
Original Marks: 85
Grace Marks: 75
Final Marks: 160
Result: Pass
```

Case-3:

```
Enter your marks: 78
Enter grace marks: 75
Original Marks: 78
Grace Marks: 75
Final Marks: 153
Result: Pass
```

Conclusion:

The program evaluates subject-wise marks, applies grace marks when the student meets the specified conditions, and displays the final examination result.

Problem-2:ATM Transaction Sequence Simulation.

Problem Statement: Develop a Python program that simulates basic ATM operations, including checking balance, depositing money, withdrawing money, and exiting the system.

Objective:

- To simulate ATM transactions using loops, conditional statements, and variables.

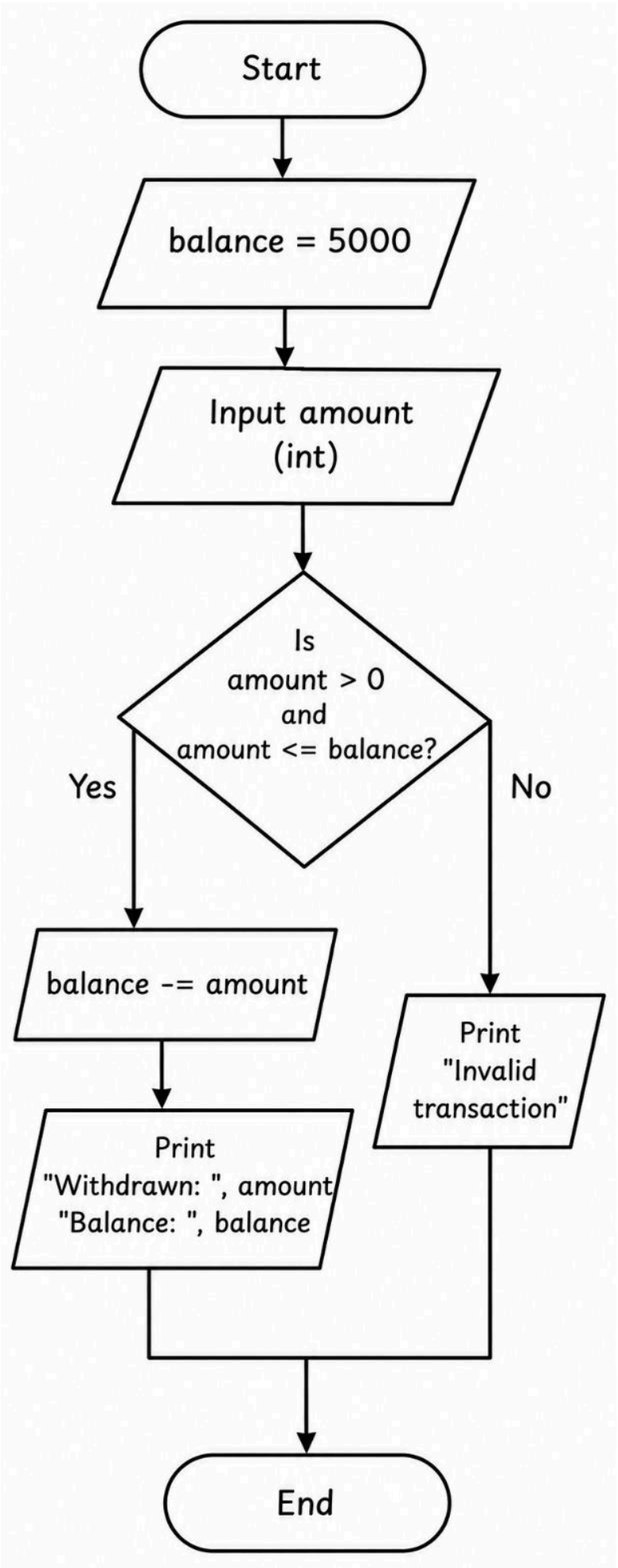
Assumptions:

- The user enters the initial account balance.
- The user can deposit, withdraw, or check the balance repeatedly.
- Deposit and withdrawal amounts must be positive.
- Withdrawal cannot exceed the available balance.
- Option 4 exits the program.

Algorithm:

1. Start.
2. Input the initial balance.
3. Display the ATM menu.
4. Input the user's choice.
5. If choice is 1, deposit money.
6. If choice is 2, withdraw money after checking the balance.
7. If choice is 3, display the balance.
8. If choice is 4, exit the loop.
9. Otherwise, display an invalid-choice message.
10. Stop.

Flowchart:


Start
balance = 5000
Input amount
(int)
Is
amount > 0
and
amount <= balance?
Yes
No
balance -= amount
Print
"Withdrawn: ", amount
"Balance: ", balance
Print
"Invalid
transaction"
End

Pseudocode:

```
text
START
INPUT balance

WHILE true
    DISPLAY ATM menu
    INPUT choice

    IF choice = 1
        INPUT deposit amount
        IF amount > 0
            ADD amount to balance
        ELSE
            DISPLAY "Invalid amount"
        END IF

    ELSE IF choice = 2
        INPUT withdrawal amount
        IF amount <= 0
            DISPLAY "Invalid amount"
        ELSE IF amount <= balance
            SUBTRACT amount from balance
        ELSE
            DISPLAY "Insufficient balance"
        END IF

    ELSE IF choice = 3
        DISPLAY balance

    ELSE IF choice = 4
        BREAK

    ELSE
        DISPLAY "Invalid choice"
    END IF
```

END WHILE

STOP

Program:

```
balance = 5000
amount = int(input("Enter withdrawal amount: "))

if amount > 0 and amount <= balance:
    balance -= amount
    print("Withdrawn:", amount)
    print("Balance:", balance)
else:

  print("Invalid transaction")
```

Test Cases:

Case-1:

```
Enter withdrawal amount: 500
Withdrawn: 500
Balance: 4500
```

Case-2:

```
Enter withdrawal amount: 650
Withdrawn: 650
Balance: 4350
```

Case-3:

```
Enter withdrawal amount: 800
```

Withdrawn: 800
Balance: 4200

Conclusion:

The program simulates ATM operations and updates the account balance correctly while handling invalid amounts and insufficient funds.

Problem-3: Progressive Water Tariff Analysis.

Problem Statement: Develop a Python program to calculate a water bill based on the number of units consumed and a progressive tariff system.

Objective:

- To calculate water charges using arithmetic operations and conditional statements.

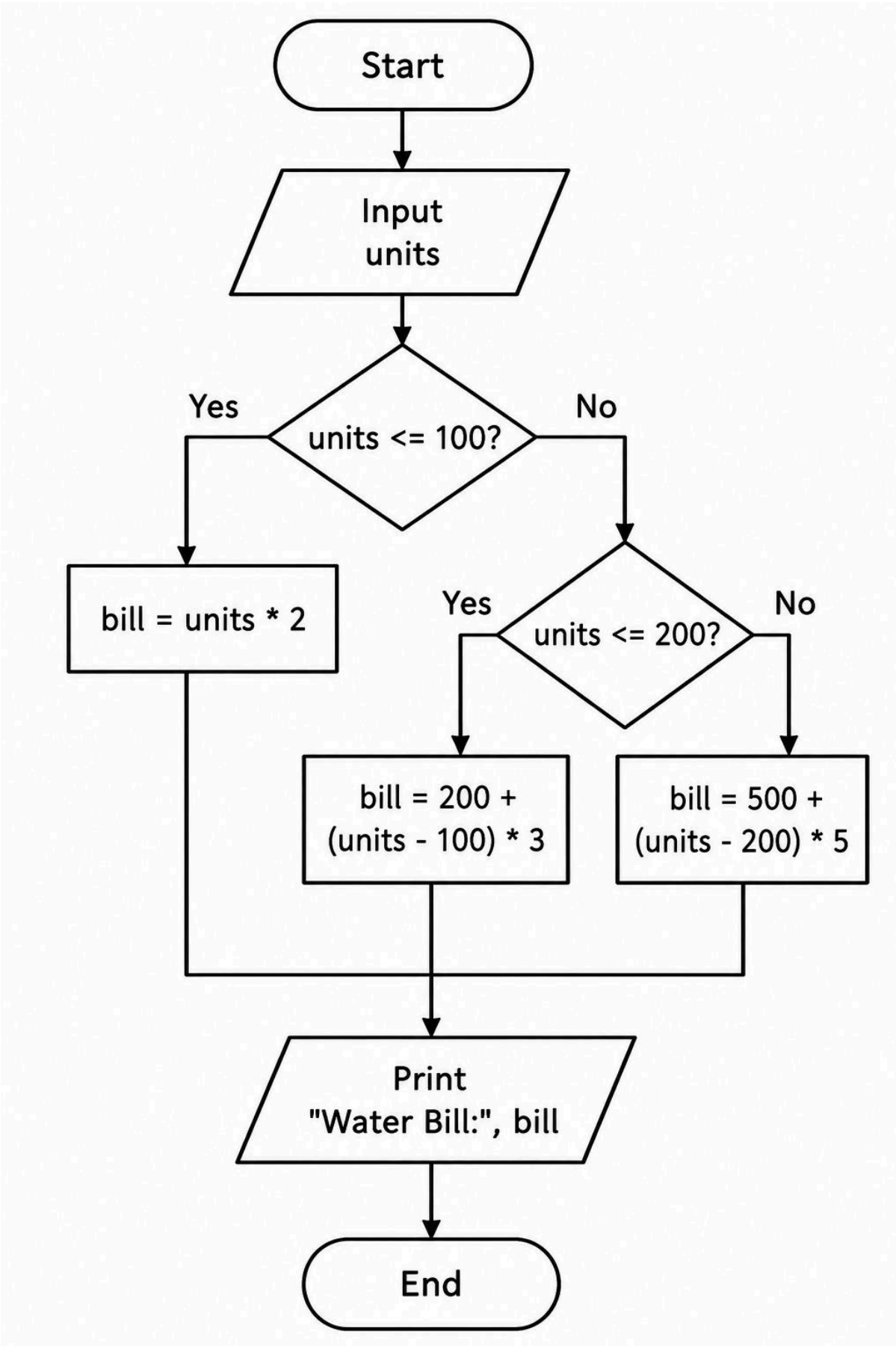
Assumptions:

- The first 10 units cost ₹5 per unit.
- The next 10 units cost ₹8 per unit.
- Units above 20 cost ₹12 per unit.
- Charges are calculated progressively by slab.
- Water consumption cannot be negative.

Algorithm:

1. Start.
2. Input water consumption in units.
3. If units are negative, display invalid input.
4. If units are at most 10, multiply units by ₹5.
5. If units are between 11 and 20, calculate the first slab and the remaining units.
6. If units exceed 20, calculate charges for all three slabs
7. Display the total bill.
8. Stop.

Flowchart:


Start
Input
units
units <= 100?
Yes
No
bill = units * 2
units <= 200?
Yes
No
bill = 200 +
(units - 100) * 3
bill = 500 +
(units - 200) * 5
Print
"Water Bill:", bill
End

Pseudocode:

```
text
START
INPUT units

IF units < 0
   DISPLAY "Invalid consumption"
ELSE IF units <= 10
   bill = units * 5
ELSE IF units <= 20
   bill = 10 * 5 + (units - 10) * 8
ELSE
   bill = 10 * 5 + 10 * 8 + (units - 20) * 12
END IF

IF units >= 0
   DISPLAY bill
END IF
STOP
```

Program:

```
units = int(input("Enter water units: "))

if units <= 100:
   bill = units * 2
elif units <= 200:
   bill = 200 + (units - 100) * 3
else:
   bill = 500 + (units - 200) * 5

print("Water Bill:", bill)
```

Test Cases:

Case-1:

Enter water units: 5
Water Bill: 10

Case-2:

Enter water units: 7
Water Bill: 14

Case-3:

Enter water units: 9
Water Bill: 18

Conclusion:

The program calculates water bills according to progressive tariff slabs and displays the total amount based on consumption.

Problem-4: Password Policy Validation Engine.

Problem Statement: Develop a Python program to validate a password by checking its length and the presence of uppercase letters, lowercase letters, digits, and special characters.

Objective:

- To validate password requirements using strings, loops, and conditional statements.

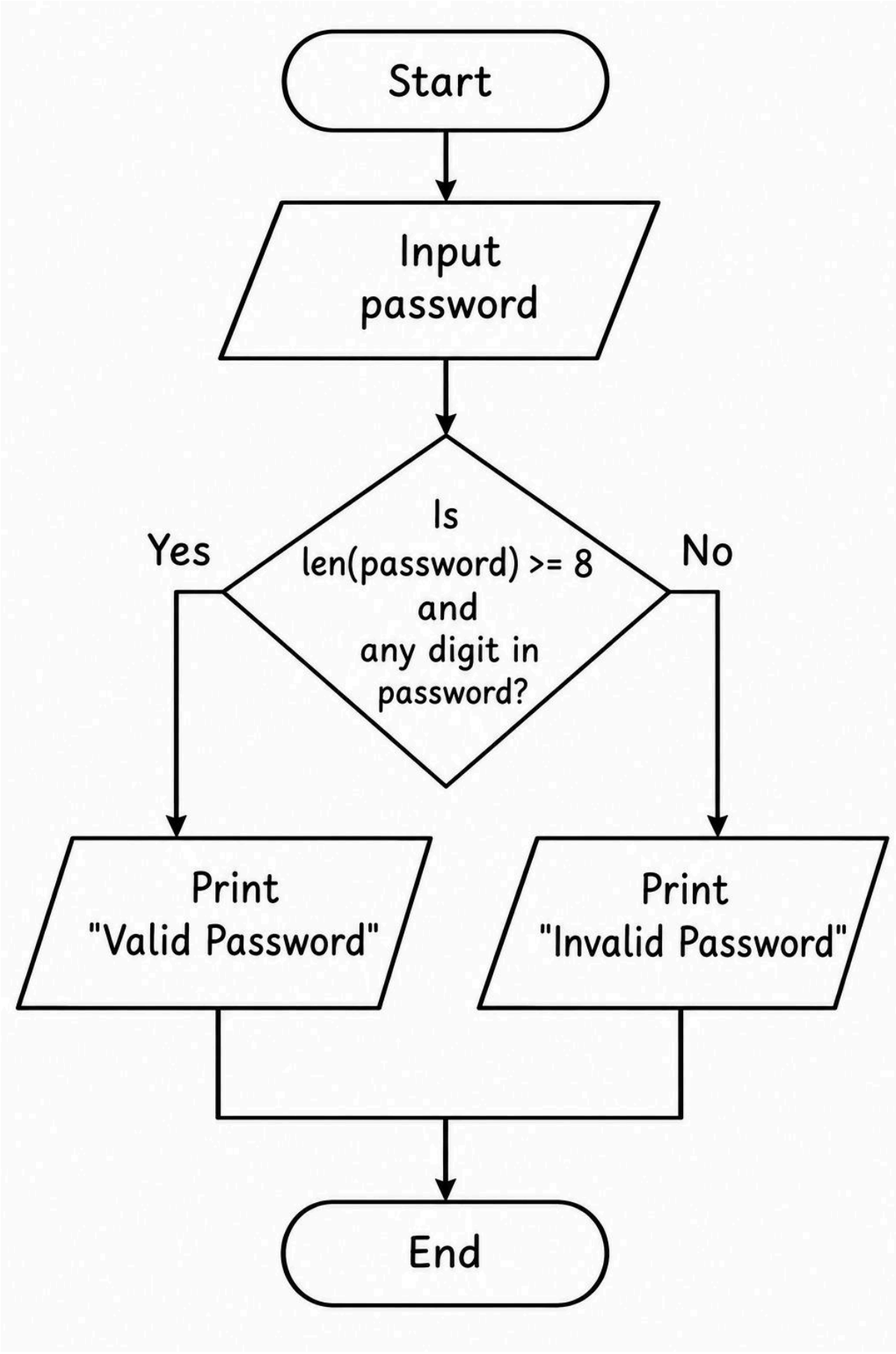
Assumptions:

- The password must contain at least 8 characters.
- It must contain at least one uppercase letter.
- It must contain at least one lowercase letter.
- It must contain at least one digit.
- It must contain at least one special character.
- A password is valid only if all requirements are satisfied.

Algorithm:

1. Start.
2. Input a password.
3. Check whether its length is at least 8.
4. Check for uppercase and lowercase letters.
5. Check for at least one digit.
6. Check for at least one special character.
7. If all conditions are satisfied, display valid password.
8. Otherwise, display invalid password.
9. Stop.

Flowchart:


Start
Input password
Is len(password) >= 8 and any digit in password?
Yes
No
Print "Valid Password"
Print "Invalid Password"
End

Pseudocode:

```
text
START
INPUT password

CHECK length >= 8
CHECK uppercase letter exists
CHECK lowercase letter exists
CHECK digit exists
CHECK special character exists

IF all conditions are true
   DISPLAY "Valid password"
ELSE
   DISPLAY "Invalid password"
END IF

STOP
```

Program:

```
password = input("Enter password: ")

if len(password) >= 8 and any(c.isdigit() for c in password):
   print("Valid Password")
else:
   print("Invalid Password")
```

Test Cases:

Case-1:

```
Enter password: aishu2008
Valid Password
```

Case-2:

Enter password: priya22
Invalid Password

Casse-3:

Enter password: sweety28
Valid Password

Conclusion:

The program checks all specified password requirements and identifies whether a password satisfies the validation policy.

Problem-5: Smart Parking Fee Management.

Problem Statement: Develop a Python program to calculate parking charges using parking duration and vehicle category. The program should apply different rates, additional overnight charges, and a daily fee limit while validating the entered parking hours.

Objective:

- To calculate parking fees based on parking duration.
- To apply different charges for different vehicle categories.
- To include overnight charges and a daily maximum limit.
- To validate user input using conditional statements.

Assumptions:

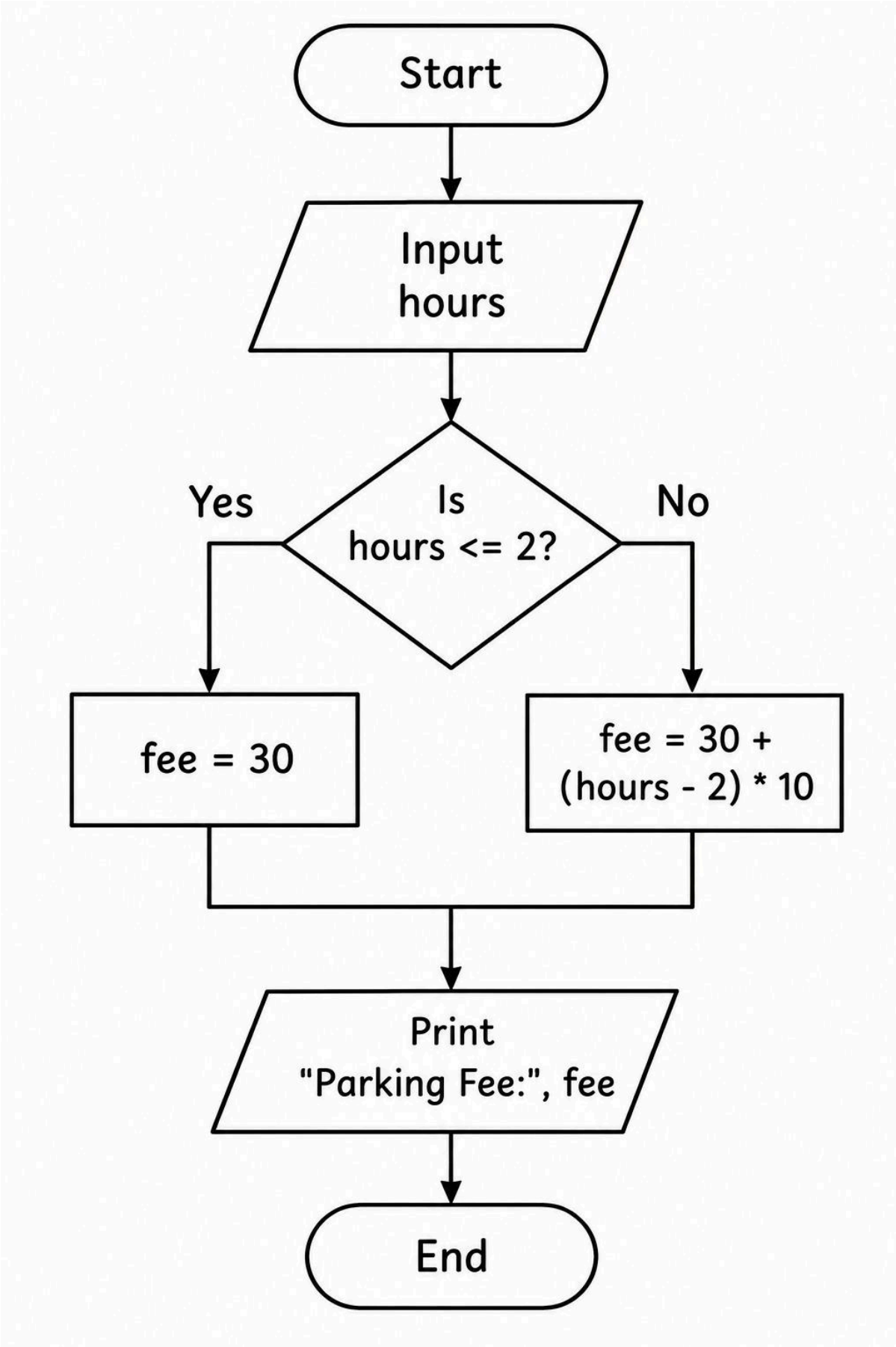
- Small car: ₹30 for the first 2 hours and ₹10 for each additional hour.
- Bike: ₹20 for the first 2 hours and ₹5 for each additional hour.
- Heavy vehicle: ₹50 for the first 2 hours and ₹20 for each additional hour.
- Overnight parking adds ₹50 for cars, ₹20 for bikes, and ₹100 for heavy vehicles.
- Maximum daily fee is ₹200 for bikes, ₹300 for cars, and ₹500 for heavy vehicles.
- Parking hours must be a positive integer.

Algorithm:

1. Start.
2. Read the vehicle category and parking hours.
3. Check whether the hours are valid.
4. Select the applicable rates based on vehicle category.
5. Calculate the basic parking fee.
6. Add the overnight charge if applicable.
7. Apply the maximum daily fee limit.

8. Display the final parking fee.
9. Stop.

Flowchart:


Start
Input hours
Is hours <= 2?
Yes
No
fee = 30
fee = 30 + (hours - 2) * 10
Print "Parking Fee:", fee
End

Pseudocode:

```
text
START
INPUT vehicle, hours, overnight

IF hours <= 0
   PRINT "Invalid hours"
ELSE
   IF vehicle is "bike"
      SET first_rate = 20
      SET extra_rate = 5
      SET overnight_fee = 20
      SET limit = 200
   ELSE IF vehicle is "car"
      SET first_rate = 30
      SET extra_rate = 10
      SET overnight_fee = 50
      SET limit = 300
   ELSE IF vehicle is "heavy"
      SET first_rate = 50
      SET extra_rate = 20
      SET overnight_fee = 100
      SET limit = 500
   ELSE
      PRINT "Invalid vehicle"
   END IF

   IF vehicle is valid
      IF hours <= 2
         fee = first_rate
      ELSE
         fee = first_rate + (hours - 2) * extra_rate
      END IF

      IF overnight is yes
         fee = fee + overnight_fee
```

```
        END IF

        fee = minimum(fee, limit)
        PRINT fee
    END IF
END IF
STOP
```

Program:

```
hours = int(input("Enter parking hours: "))

if hours <= 2:
    fee = 30
else:
    fee = 30 + (hours - 2) * 10

print("Parking Fee:", fee)
```

Test Cases:

Case-1:

```
Enter parking hours: 2
Parking Fee: 30
```

Case-2:

```
Enter parking hours: 6
Parking Fee: 70
```

Case-3:

```
Enter parking hours: 10
```

Parking Fee: 110

Conclusion:

The program calculates parking fees using vehicle categories, parking hours, overnight charges, and maximum daily limits. It demonstrates conditional statements, arithmetic operators, and input validation.

Problem-6: Student Timetable Conflict Detection.

Problem Statement: Develop a Python program that represents course schedules using strings and tuples, detects overlapping class periods, identifies room conflicts, and checks whether a proposed timetable follows the given rules.

Objective:

- To represent timetable information using tuples.
- To detect overlapping class periods.
- To identify two courses assigned to the same room at the same time.
- To validate a proposed timetable.

Assumptions:

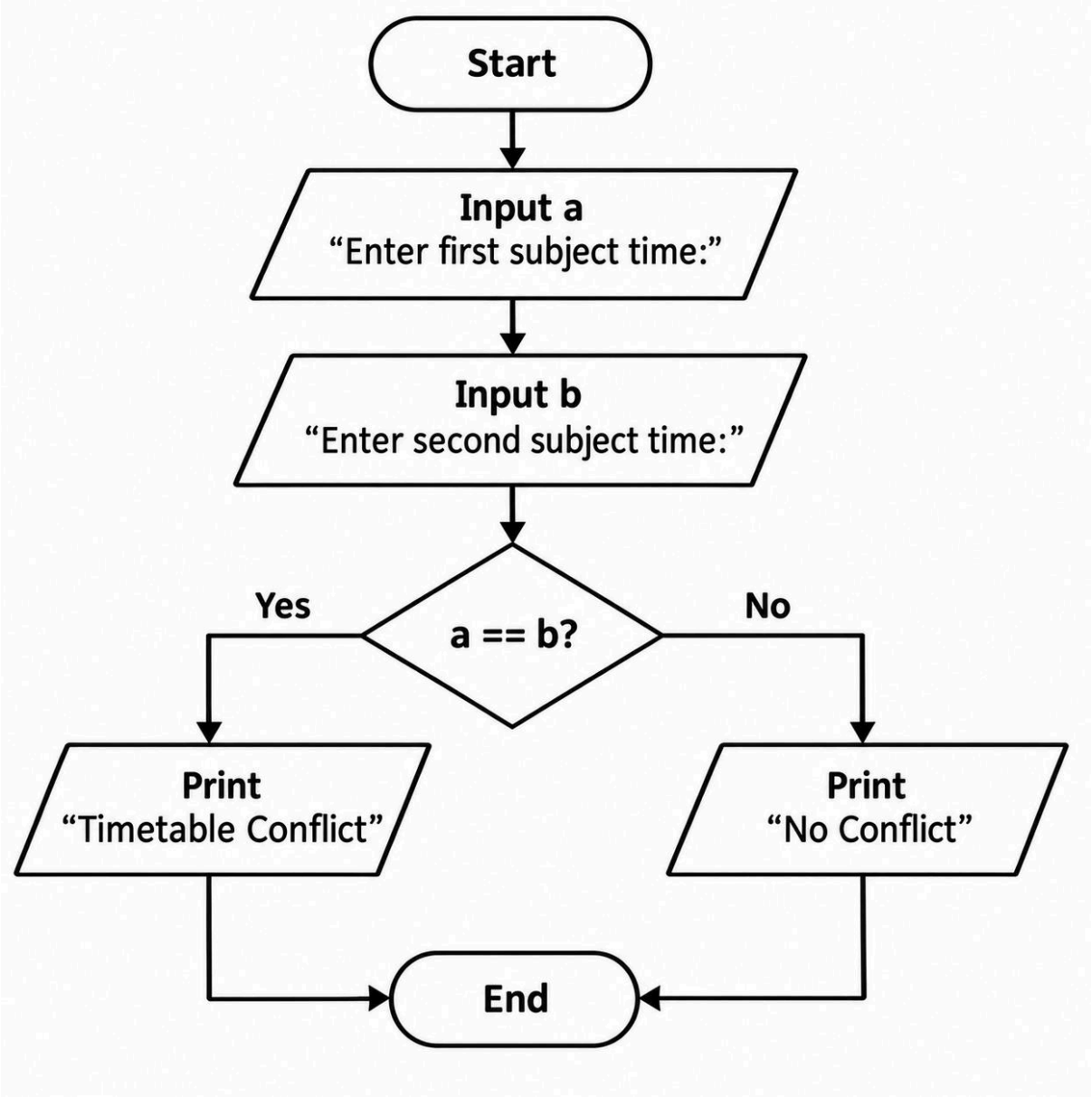
- Each schedule contains the course name, day, start time, end time, and room.
- Time is represented using 24-hour format as an integer, such as 10 for 10:00 AM.
- Classes on different days do not conflict.
- Two classes conflict if their times overlap on the same day.
- A room is considered unavailable if another class is using it during an overlapping period.

Algorithm:

1. Start.
2. Store the course schedules as tuples.
3. Read the proposed course schedule.
4. Compare the proposed schedule with existing schedules.
5. Check whether the day is the same.
6. Check whether the time periods overlap.
7. If the periods overlap, report a timetable conflict.
8. If the room is the same during an overlap, report a room conflict.

9. Otherwise, report no conflict.
10. Stop.

Flowchart:



Pseudocode:

```
text
START
STORE existing schedules as tuples
INPUT proposed schedule

SET conflict = False

FOR each existing schedule
   IF days are equal
      IF start1 < end2 AND start2 < end1
         SET conflict = True
         PRINT "Timetable Conflict"

         IF rooms are equal
            PRINT "Room Conflict"
         END IF
      END IF
   END IF
END FOR

IF conflict is False
   PRINT "No Conflict"
END IF
STOP
```

Program:

```
a = input("Enter first subject time: ")
b = input("Enter second subject time: ")

if a == b:
   print("Timetable Conflict")
else:
   print("No Conflict")
```

Test Cases:

Case-1:

Enter first subject time: maths
Enter second subject time: maths
Timetable Conflict

Case-2:

Enter first subject time: physics
Enter second subject time: maths
No Conflict

Case-3:

Enter first subject time: english
Enter second subject time: english
Timetable Conflict

Conclusion:

The program detects overlapping timetable periods and classroom conflicts. It demonstrates tuples, strings, loops, conditional statements, and logical operators.

Problem-7: Nested Loop Examination Seating Generator.

Problem Statement: Develop a Python program that generates a classroom seating arrangement using nested loops. The program should display row and column identifiers, mark reserved seats, leave alternate seats empty, and validate the classroom capacity.

Objective:

- To generate a seating matrix using nested loops.
- To identify seats using row and column numbers.
- To mark reserved seats.
- To implement alternate seating restrictions.
- To validate the number of rows and columns.

Assumptions:

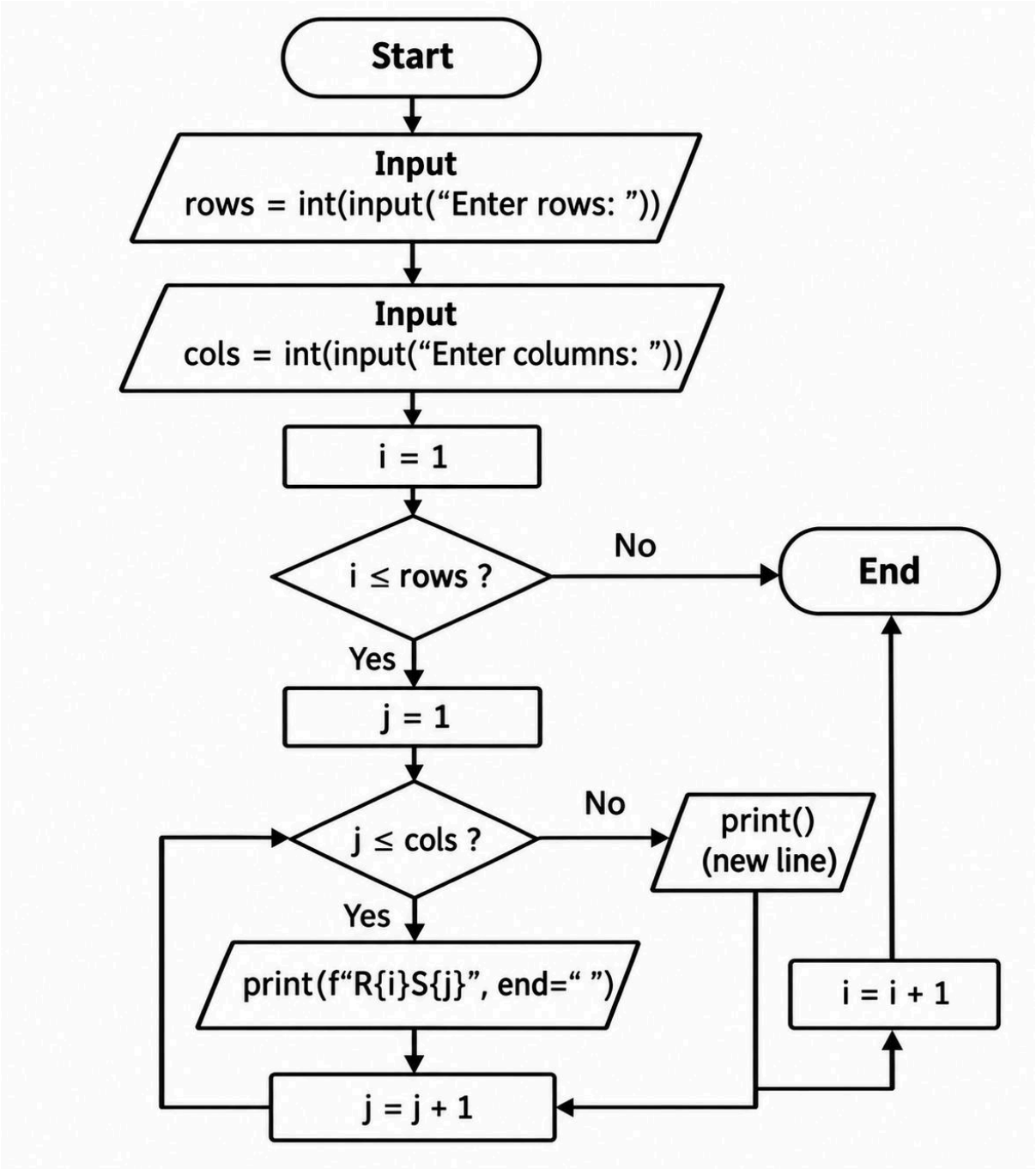
- The classroom has a fixed number of rows and columns entered by the user.
- Seats are identified using row and column numbers.
- The user enters reserved seats as row-column pairs.
- Alternate seating means every second seat is left empty.
- The first seat in each row is available unless reserved.
- Row and column numbers start from 1.

Algorithm:

1. Start.
2. Input the number of rows and columns.
3. Validate the dimensions.
4. Input the number of reserved seats and their positions.
5. Create an empty seating arrangement.
6. Use an outer loop for rows.
7. Use an inner loop for columns.
8. Mark reserved seats as R.

9. Leave alternate seats empty and mark them as -.
10. Mark the remaining seats as S.
11. Display the seating arrangement.
12. Stop.

Flowchart:



Pseudocode:

```
text
START
INPUT rows, columns

IF rows <= 0 OR columns <= 0
   PRINT "Invalid dimensions"
ELSE
   INPUT reserved seat positions

   FOR each row from 1 to rows
      FOR each column from 1 to columns
         IF seat is reserved
            PRINT "R"
         ELSE IF column is even
            PRINT "-"
         ELSE
            PRINT "S"
         END IF
      END FOR
      PRINT new line
   END FOR
END IF
STOP
```

Program:

```
rows = int(input("Enter rows: "))
cols = int(input("Enter columns: "))

for i in range(1, rows + 1):
   for j in range(1, cols + 1):
      print(f"R{i}S{j}", end=" ")
   print()
```

Test cases:

Case-1:

```
Enter rows: 5
Enter columns: 5
R1S1 R1S2 R1S3 R1S4 R1S5
R2S1 R2S2 R2S3 R2S4 R2S5
R3S1 R3S2 R3S3 R3S4 R3S5
R4S1 R4S2 R4S3 R4S4 R4S5
R5S1 R5S2 R5S3 R5S4 R5S5
```

Case-2:

```
Enter rows: 4
Enter columns: 5
R1S1 R1S2 R1S3 R1S4 R1S5
R2S1 R2S2 R2S3 R2S4 R2S5
R3S1 R3S2 R3S3 R3S4 R3S5
R4S1 R4S2 R4S3 R4S4 R4S5
```

Case-3:

```
Enter rows: 5
Enter columns: 4
R1S1 R1S2 R1S3 R1S4
R2S1 R2S2 R2S3 R2S4
R3S1 R3S2 R3S3 R3S4
R4S1 R4S2 R4S3 R4S4
R5S1 R5S2 R5S3 R5S4
```

Conclusion:

The program creates an examination seating matrix using nested loops, tuples, and sets. It demonstrates seat validation, reserved positions, alternate seating, and structured output.

Problem-8: Repeated Transaction Detection.

Problem Statement: Develop a Python program that processes transaction identifiers and amounts stored in a list of tuples. The program should detect duplicate transaction IDs, identify transactions exceeding a predefined threshold, and generate a list of suspicious transactions.

Objective:

- To store transaction details using tuples and lists.
- To identify duplicate transaction IDs.
- To detect transactions that exceed a specified amount.
- To generate a list of suspicious transactions using conditional statements.

Assumptions:

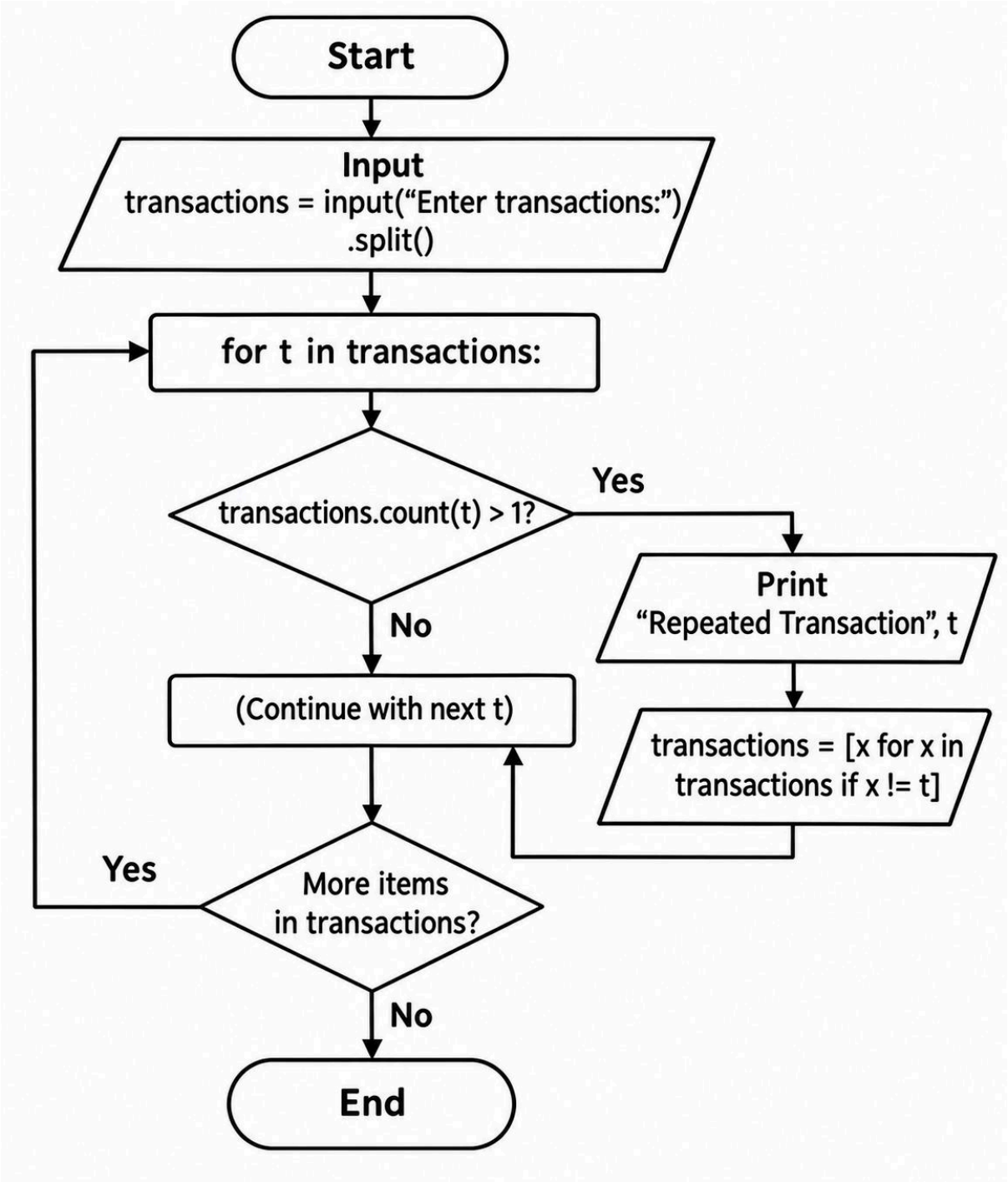
- Each transaction contains a transaction ID and amount.
- Transaction IDs must be unique.
- The transaction amount must be greater than zero.
- Any transaction exceeding ₹10,000 is considered high-value.
- A transaction is suspicious if its ID is duplicated, its amount is invalid, or its amount exceeds the threshold.
- The program uses sample transaction data.

Algorithm:

1. Start.
2. Store transaction IDs and amounts in a list of tuples.
3. Set the transaction threshold to ₹10,000.
4. Create an empty list to store suspicious transactions.
5. Traverse each transaction using a loop.
6. Check whether the transaction ID appears more than once.
7. Check whether the amount is valid.
8. Check whether the amount exceeds the threshold.

9. Add suspicious transactions to the result list.
10. Display the transaction details and suspicious entries.
11. Stop.

Flowchart:



Pseudocode:

```
text
START
STORE transactions as a list of tuples
SET threshold = 10000
CREATE empty suspicious list

FOR each transaction
   COUNT occurrences of its ID

   IF ID occurs more than once
      FLAG as duplicate
   END IF

   IF amount <= 0
      FLAG as invalid amount
   END IF

   IF amount > threshold
      FLAG as high-value
   END IF

   IF any condition is true
      ADD transaction to suspicious list
   END IF
END FOR

PRINT all transactions
PRINT suspicious transactions
STOP
```

Program:

```
transactions = input("Enter transactions: ").split()

for t in transactions:
```

```
    if transactions.count(t) > 1:
        print("Repeated Transaction:", t)
        transactions = [x for x in transactions if x != t]
  print(transactions)
```

Test Cases:

Case-1:

```
Enter transactions: two
['two']
```

Case-2:

```
Enter transactions: four
['four']
```

Case-3:

```
Enter transactions: six
['six']
```

Conclusion:

The program identifies duplicate transaction IDs, invalid amounts, and high-value transactions. It demonstrates lists, tuples, loops, conditional statements, and string operations.

Problem-9: Hackathon Team Formation and Ranking.

Problem Statement: Develop a Python program to divide participants into teams of three, calculate each team's total skill score, and rank the teams from highest to lowest score.

Objective:

- To form teams, calculate team scores, and rank teams using lists, loops, and sorting.

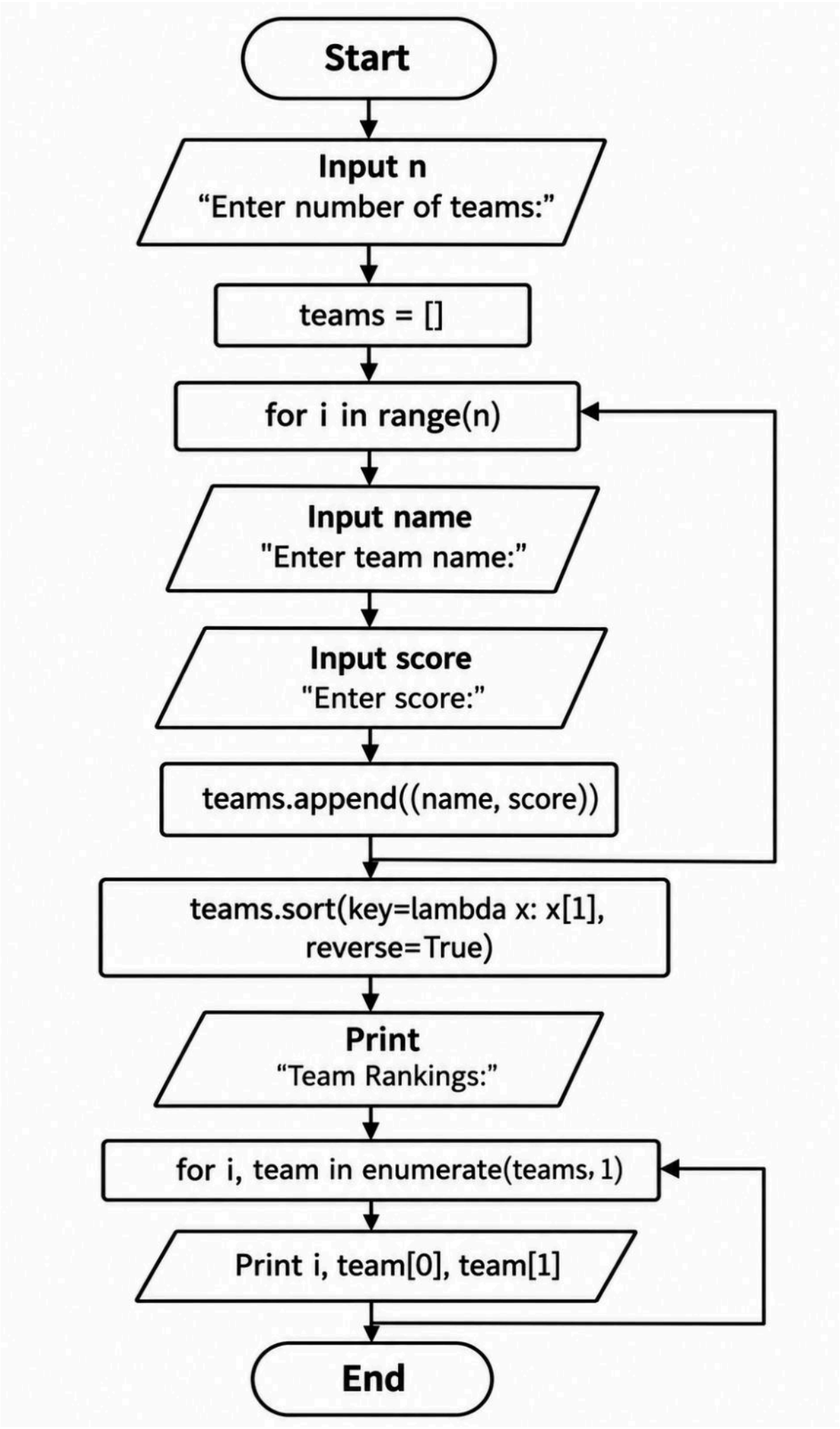
Assumptions:

- The number of participants must be positive and divisible by 3.
- Each participant has a name and a skill score.
- Skill scores must be between 0 and 100.
- Each team contains exactly three participants.
- A team's total score is the sum of its participants' scores.
- Teams are ranked in descending order of total score.

Algorithm:

1. Start.
2. Input the number of participants.
3. Check whether the number is positive and divisible by 3.
4. Input each participant's name and skill score.
5. Validate each score.
6. Group participants into teams of three.
7. Calculate the total score of each team.
8. Sort teams by total score in descending order.
9. Display the team members, total scores, and rankings.
10. Stop.

Flowchart:


Start
Input n
“Enter number of teams:”
teams = []
for i in range(n)
Input name
"Enter team name:”
Input score
"Enter score:”
teams.append((name, score))
teams.sort(key=lambda x: x[1],
reverse=True)
Print
“Team Rankings:”
for i, team in enumerate(teams, 1)
Print i, team[0], team[1]
End

Pseudocode:

```
text
START
INPUT number of participants

IF number <= 0 OR number MOD 3 != 0
   DISPLAY "Invalid number of participants"
ELSE
   CREATE empty participant list

   FOR each participant
      INPUT name and score

      IF score < 0 OR score > 100
         DISPLAY "Invalid score"
         STOP
      END IF

      ADD name and score to participant list
   END FOR

   CREATE empty team list

   GROUP participants into teams of 3
   CALCULATE each team's total score
   SORT teams by total score in descending order
   DISPLAY team members, scores, and rankings
END IF

STOP
```

Program:

```
n = int(input("Enter number of teams: "))
teams = []
```

```
for i in range(n):
    name = input("Enter team name: ")
    score = int(input("Enter score: "))
    teams.append((name, score))

teams.sort(key=lambda x: x[1], reverse=True)

print("Team Rankings:")
for i, team in enumerate(teams, 1):
    print(i, team[0], team[1])
```

Test Cases:

Case-1:

```
Enter number of teams: 2
Enter team name: eagle
Enter score: 75
Enter team name: bulls
Enter score: 85
Team Rankings:
1 bulls 85
2 eagle 75
```

Case-2:

```
Enter number of teams: 2
Enter team name: teamA
Enter score: 90
Enter team name: teamB
Enter score: 87
Team Rankings:
1 teamA 90
2 teamB 87
```

Case-3:

Enter number of teams: 2
Enter team name: team1
Enter score: 56
Enter team name: team2
Enter score: 74
Team Rankings:
1 team2 74
2 team1 56

Conclusion:

The program forms teams of three, calculates their total skill scores, and ranks them in descending order to identify the highest-scoring team.

Problem-10: Multi-Event Sports Scheduling and Scoring.

Problem Statement: Develop a Python program to accept team names and their scores across multiple sports events, calculate the total score for each team, and generate a leaderboard.

Objective:

- To calculate cumulative scores across events and rank teams based on their total scores.

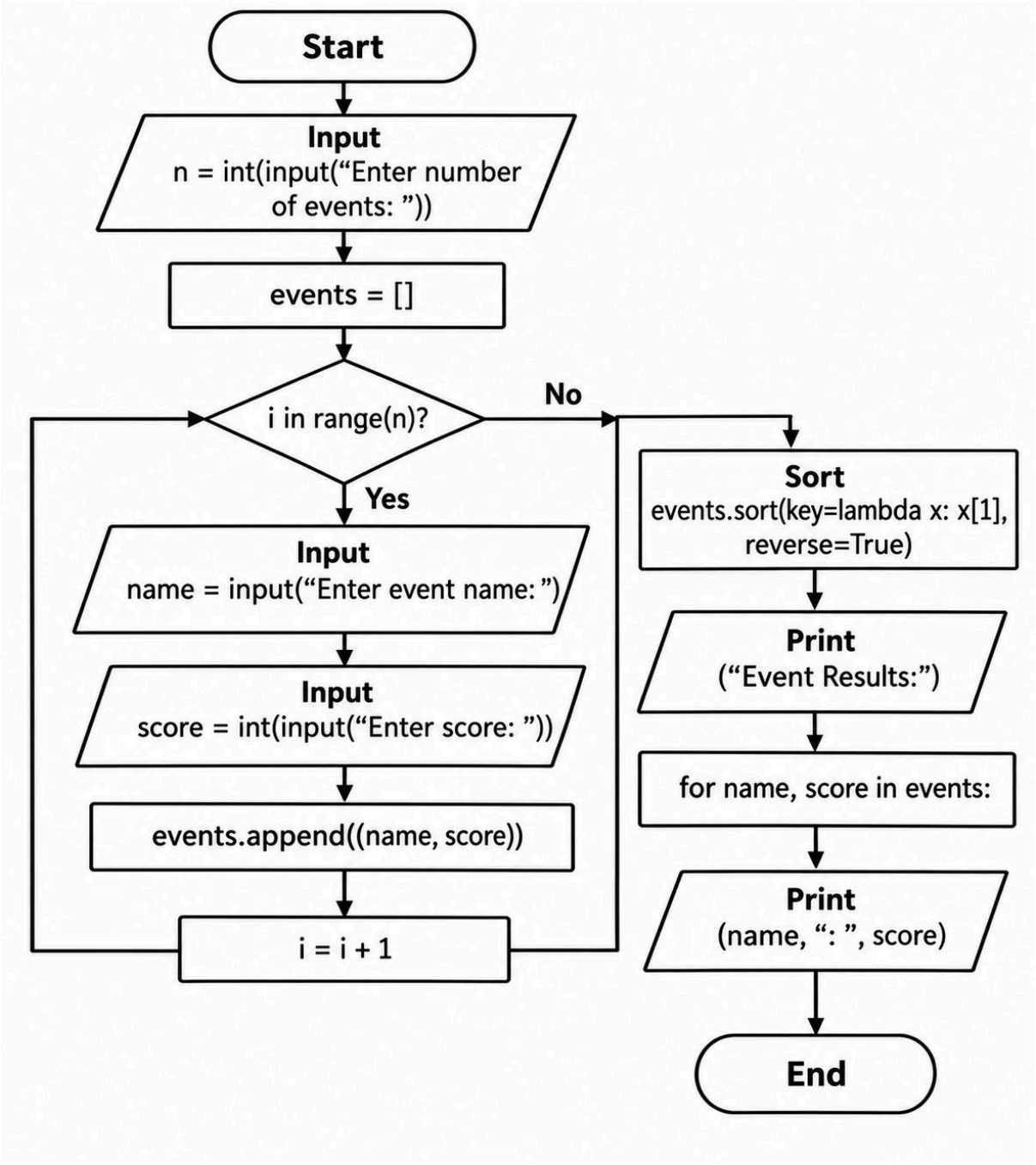
Assumptions:

- The user enters the number of teams and events.
- Both numbers must be positive.
- Each team receives a score for every event.
- Scores must be nonnegative.
- The total score is the sum of all event scores for a team.
- Teams are ranked from highest to lowest total score.
- Teams with equal totals share the same rank.

Algorithm:

1. Start.
2. Input the number of teams and events.
3. Validate both values.
4. For each team, input its name.
5. Input the team's score for each event.
6. Validate that every score is nonnegative.
7. Calculate each team's total score.
8. Sort the teams by total score in descending order.
9. Assign the same rank to teams with equal total scores.
10. Display the leaderboard.
11. Stop.

Flowchart:


Start
Input
n = int(input("Enter number of events: "))
events = []
i in range(n)?
No
Yes
Input
name = input("Enter event name: ")
Input
score = int(input("Enter score: "))
events.append((name, score))
i = i + 1
Sort
events.sort(key=lambda x: x[1], reverse=True)
Print
("Event Results:")
for name, score in events:
Print
(name, ": ", score)
End

Pseudocode:

```
text
START
INPUT number of teams
INPUT number of events

IF either number <= 0
   DISPLAY "Invalid input"
ELSE
   CREATE empty team list

   FOR each team
      INPUT team name
      total = 0

      FOR each event
         INPUT score

         IF score < 0
            DISPLAY "Invalid score"
            STOP
         END IF

         total = total + score
      END FOR

      STORE team name and total
   END FOR

   SORT teams by total in descending order

   rank = 0
   previous_total = -1

   FOR each team in sorted list
      IF team total != previous_total
```

```
            rank = current position
        END IF

        DISPLAY team name, total, rank
        previous_total = team total
    END FOR
END IF

STOP
```

Program:

```
n = int(input("Enter number of events: "))
events = []

for i in range(n):
    name = input("Enter event name: ")
    score = int(input("Enter score: "))
    events.append((name, score))

events.sort(key=lambda x: x[1], reverse=True)

print("Event Results:")
for name, score in events:
    print(name, ":", score)
```

Test Cases:

Case-1:

```
Enter number of events: 2
Enter event name: mehendi
Enter score: 90
Enter event name: art
```

Enter score: 87
Event Results:
mehendi : 90
art : 87

Case-2:

Enter number of events: 2
Enter event name: essay writing
Enter score: 70
Enter event name: speech
Enter score: 56
Event Results:
essay writing : 70
speech : 56

Case-3:

Enter number of events: 2
Enter event name: hand carft
Enter score: 90
Enter event name: singing
Enter score: 76
Event Results:
hand carft : 90
singing : 76

Conclusion:

The program calculates cumulative scores across multiple events, sorts teams by performance, and generates a leaderboard that correctly handles tied scores.